

"The best way to predict the future is to create it."



FROM ZERO TO CONFIDENT



LINUX NOTES

FOR BEGINNERS

One command at a time,
One step closer to mastery.



- ✓ Commands
- ✓ Examples
- ✓ Concepts
- ✓ Practice
- ✓ Cheat Sheets

```
$ learn  
$ practice  
$ repeat  
$ succeed  
$ _
```

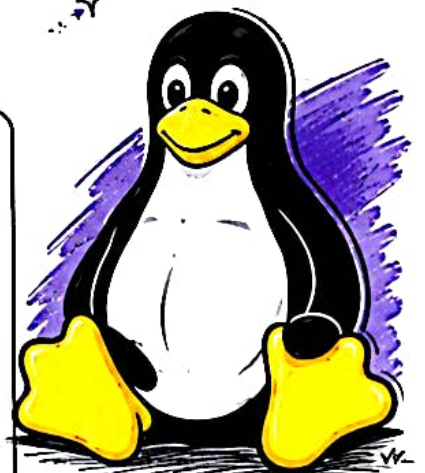
10 HANDWRITTEN PAGES OF
ESSENTIAL LINUX KNOWLEDGE

PERFECT FOR

- Beginners
- Students
- DevOps Engineers
- System Admins
- Anyone who wants to level up!

WHAT'S INSIDE?

 INTRODUCTION TO LINUX	 FILE SYSTEM	 NAVIGATION COMMANDS	 FILE MANAGEMENT	 FILE VIEWING COMMANDS
 SEARCH COMMANDS	 PERMISSIONS & OWNERSHIP	 USERS & GROUPS	 PROCESSES & JOBS	 PACKAGE MANAGEMENT



LEARN.
PRACTICE.
BUILD.
SUCCEED.

SIMPLE NOTES | PRACTICAL EXAMPLES | REAL WORLD USE



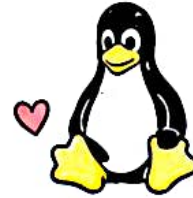
The more you practice today,
the easier tomorrow gets.



♥ SAVE THIS POST & SHARE WITH SOMEONE WHO IS LEARNING LINUX!

1

LINUX INTRODUCTION & FILE SYSTEM

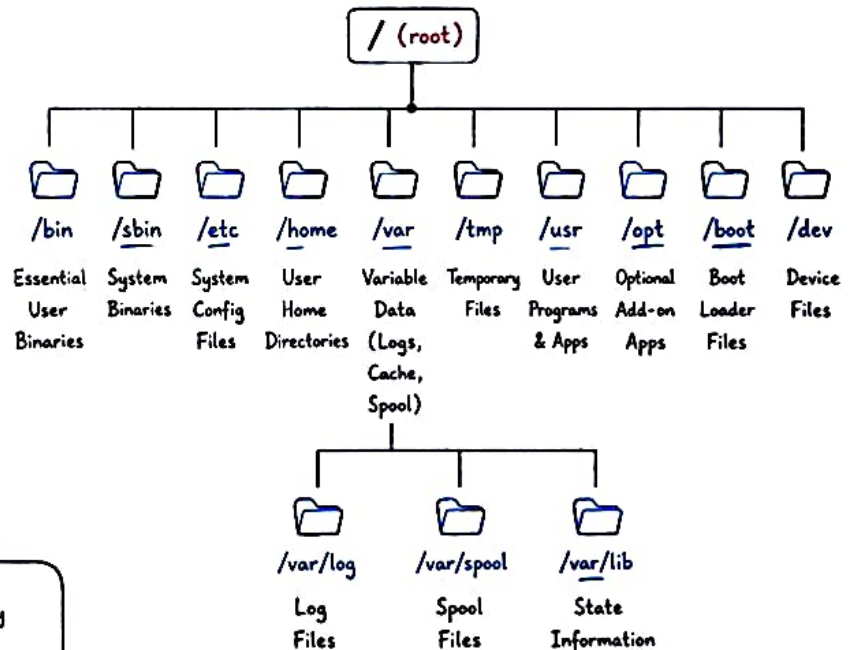


In Linux,
everything
is a file.

WHAT IS LINUX?

- Linux is an open source, free and powerful operating system.
 - It is widely used in Servers, Cloud, DevOps, Embedded systems and more.
 - Everything in Linux is treated as a file.
- It is Secure, Stable and Highly Customizable.

LINUX FILE SYSTEM STRUCTURE



IMPORTANT DIRECTORIES

/	: Root directory. Top of the hierarchy
/bin	: Basic user command binaries
/sbin	: System admin binaries
/etc	: Configuration files
/home	: Users' home directories
/var	: Variable data like logs, cache, spool
/usr	: User installed programs & libraries
/tmp	: Temporary files (cleared on reboot)
/opt	: Optional third-party software
/boot	: Boot loader files
/dev	: Device files
/proc	: Virtual filesystem with process info
/sys	: System information (kernel)
/lib	: Shared libraries
/mnt	: Mount point for temporary file systems
/media	: External media mount point (USB, CD)
/root	: Root user's home directory



HANDS-ON EXAMPLES

<code>pwd</code>	- Print working directory
<code>ls -l /</code>	- List root directory in long format
<code>cd /etc</code>	- Change directory to /etc
<code>ls -l /home</code>	- List all users' home directories
<code>df -h</code>	- Check disk usage
<code>du -sh /var/log</code>	- Check size of log directory
<code>tree /usr head -n 20</code>	- Explore directory tree



KEY POINTS TO REMEMBER

- ✓ Everything starts from root /
- ✓ Linux is case-sensitive (File ≠ file)
- ✓ Use Tab key for auto-completion
- ✓ Use `man <command>` to read manual
- ✓ Practice in a Linux environment daily



Pro Tip:

A strong Linux foundation makes you 10x better in DevOps!

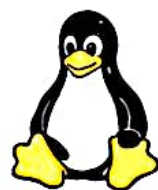
SUMMARY

Linux is the backbone of DevOps.
Understand the file system, explore directories and practice basic commands every day.
That's how you become confident!



2

LINUX NAVIGATION & DIRECTORY MANAGEMENT



In Linux,
where you are
matters!

1 UNDERSTANDING PATHS

- **Absolute Path** : Starts from root (/) and shows the full location.
Ex: /home/user/docs/file.txt
- **Relative Path** : Starts from current directory and shows the location relative to it.
Ex: docs/file.txt



Tip:

Use **Tab** key for auto-completion of file and directory names.

2 ESSENTIAL NAVIGATION COMMANDS

	pwd	Print working directory (Shows your current location)	\$ pwd /home/user
	cd	Change directory	\$ cd /etc \$ cd .. (go back) \$ cd ~ (go to home)
	ls	List files and directories	\$ ls \$ ls -l \$ ls -lah
	tree	Display directory tree (You may need to install it)	\$ tree .
	stat	Show file or directory status	\$ stat file.txt

3 HIDDEN FILES

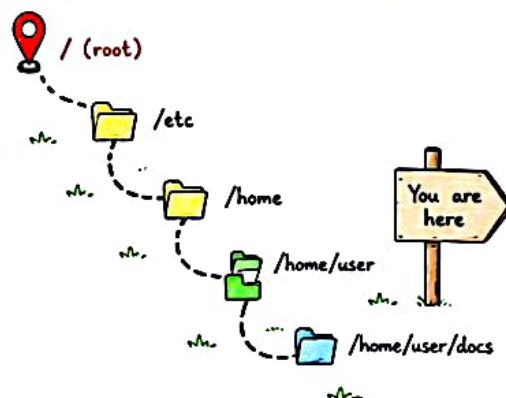
Files or directories starting with a dot (.) are hidden.

```
$ ls -a
.  ..  .bashrc  .config  file.txt
```

- . → Current directory
- .. → Parent directory

4 NAVIGATION SHORTCUTS

- . → Current directory
- .. → Parent directory
- ~ → Home directory
- / → Root directory
- - → Previous directory



5 PRACTICAL EXAMPLES

```
$ pwd          # Show current location
$ ls -lah      # List all files with details
$ cd /var/log  # Go to /var, log directory
ls            # List files in /var/log
$ cd ..        # Go back to /var
$ cd ~        # Go to home directory
$ tree -L 2    # Show directory tree up to 2 levels
```

6 COMMON MISTAKES

- ✗ Using **rm *** in wrong directory
- ✗ Not checking path before deleting files
- ✗ Confusing absolute and relative paths
- ✗ Forgetting hidden files (.)



7 MINI CHALLENGE

1. Go to /etc directory.
2. List all .conf files with details.
3. Create a directory named practice in your home.
4. Go inside it and create 2 empty files.



SUMMARY

- Linux navigation is all about knowing where you are and how to move.
- Master these commands and you can explore the system with confidence.
- Practice daily - small steps, big results!



3

LINUX FILE MANAGEMENT



In Linux, everything is a file!



CONCEPT

Linux treats all data as files. These commands help you create, copy, move, rename and delete files and directories efficiently.



TYPES OF FILES



Regular File



Directory (Folder)



Symbolic Link



Device File



And More

ESSENTIAL COMMANDS

COMMAND	DESCRIPTION	EXAMPLE	OUTPUT / RESULT
touch	Create an empty file or update timestamp	\$ touch notes.txt	Creates file: notes.txt
mkdir	Create a new directory	\$ mkdir project	Creates directory: project/
cp	Copy files or directories	\$ cp file.txt backup.txt \$ cp -r dir1 dir2	Copies file.txt to backup.txt Copies dir1 to dir2 (recursively)
mv	Move or rename files/directories	\$ mv old.txt new.txt \$ mv file.txt /home/user/	Renames old.txt to new.txt Moves file.txt to /home/user/
rm	Remove (delete) files	\$ rm file.txt \$ rm -i file.txt	Deletes file.txt Asks before deleting
rmdir	Remove an empty directory	\$ rmdir empty_dir	Deletes empty_dir directory

FILE NAMING RULES

- Case-sensitive (file.txt and File.txt are different)
- Spaces allowed (use quotes: "my file.txt")
- Special characters allowed, but use with care
- Avoid: / \ : * ? " < > |
- No limit on length (but keep it meaningful)



PRACTICAL EXAMPLES

```
$ touch a.txt           # Create file
$ mkdir -p a/b/c        # Create nested dirs
$ cp a.txt b.txt        # Copy file
$ cp -r a/ b/           # Copy directory
$ mv a.txt docs/        # Move file
$ mv old.txt new.txt    # Rename file
$ rm temp.txt           # Delete file
$ rmdir empty_dir       # Remove empty dir
```

WILDCARDS (USEFUL!)

- * → matches any number of characters
Ex: rm *.log (deletes all .log files)
- ? → matches any single character
Ex: ls file?.txt (file1.txt, fileA.txt)
- [abc] → matches any one character in brackets
Ex: ls file[0-9].txt



COMMON MISTAKES

- rm * → Can delete everything in current directory!
- rm -rf / → Never run this command!
- Not checking path before rm
- Using rmdir on non-empty directory
- Overwriting files without backup



PRACTICE TASK



- 1 Create a directory named work.
- 2 Inside it, create three files: a.txt, b.txt, c.txt
- 3 Copy a.txt to backup_a.txt
- 4 Move b.txt to docs/ directory
- 5 Rename c.txt to notes.txt
- 6 Delete backup_a.txt

Try these in your terminal! 😊

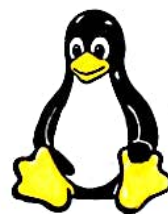
SUMMARY

- ✓ Linux provides powerful commands to manage files and directories.
- ✓ Practice these commands daily.
- ✓ Small steps today, strong skills tomorrow!



4

LINUX VIEWING & EDITING FILES



In Linux, files are meant to be read, understood and updated!

1 WHY DO WE VIEW FILES?

- To read configuration files
- To check logs and outputs
- To understand system, applications and settings
- To debug and troubleshoot



2 COMMON COMMANDS OVERVIEW

COMMAND	DESCRIPTION	BEST FOR	EXAMPLE
cat	Display entire file content	Small files	cat file.txt
less	View file page by page (scroll)	Large files	less file.log
more	View file page by page (one direction)	Large files (simple browsing)	more file.log
head	Show first N lines	Check start of a file	head -n 10 file.txt
tail	Show last N lines	Check end of a file / logs	tail -n 20 file.log
nl	Number all lines of a file	Read with line numbers	nl file.txt

3 WHEN TO USE WHAT?

- ✓ **cat** → Small files, quick output
- ✓ **less** → Large files, need scrolling
- ✓ **more** → Older systems, one-way scroll
- ✓ **head** → Check top of logs/config
- ✓ **tail** → Monitor log files (end part)
- ✓ **nl** → Read file with line numbers



4 EDITING FILES

nano (Beginner Friendly)

- Simple and easy to use
- Good for quick edits

```
$ nano file.txt
```

Basic Shortcuts

- ^O → Save
- ^X → Exit
- ^K → Cut line
- ^U → Paste line
- ^W → Search
- (^ = Ctrl)

vim (Powerful Editor)

- Advanced text editor
- Works in modes (command, insert...)

```
$ vim file.txt
```

Basic Modes & Keys

- | COMMAND MODE | INSERT MODE |
|-------------------|----------------------------|
| i → Insert mode | ESC → Back to command mode |
| :w → Save | |
| :q → Quit | |
| :wq → Save & Quit | |
| dd → Delete line | |

5 REAL EXAMPLES

```
# View full content
$ cat config.txt

# View with scrolling
$ less system.log

# Check first 5 lines
$ head -n 5 app.log

# Check last 10 lines
$ tail -n 10 app.log

# Number all lines
$ nl notes.txt
```



6 PRACTICAL TIP

For logs, combine tail with -f to watch live updates!

```
$ tail -f /var/log/syslog
```

(Press Ctrl + C to stop)

7 COMMON MISTAKES

- ✗ Using cat on large files (use less)
- ✗ Forgetting to quit less (press 'q')
- ✗ Editing important files without backup
- ✗ Overwriting files accidentally
- ✗ Not checking file permissions before editing

8 INTERVIEW QUESTION

Q. Which command will you use to continuously monitor a log file?

Answer:

```
tail -f filename
```



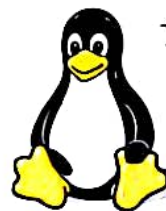
9 SUMMARY

- ✓ Use cat for quick and small files.
- ✓ Use less for large files and better control.
- ✓ head and tail are super useful for logs.
- ✓ nano is simple; vim is powerful.
- ✓ Practice daily to become comfortable!

Read more.
Understand better.
Become a Linux Pro!



5 LINUX SEARCHING & FINDING FILES



In Linux, you can search anything anywhere, anytime!

1 WHY SEARCH?

- Quickly find files and directories in large systems.
- Locate logs, configs, and data.
- Save time and boost productivity.
- Essential for troubleshooting and automation.



2 ESSENTIAL SEARCH COMMANDS

COMMAND	DESCRIPTION	BEST FOR	EXAMPLE
find	Search for files and directories by name, size, type, time, etc.	Powerful & advanced search	find /home -name file.txt
locate	Find files using updated database	Fast search (by name)	locate file.txt
grep	Search text patterns inside files	Search content (patterns)	grep "error" app.log
which	Find the path of an executable command	Find commands in PATH	which python3
whereis	Locate binary, source and man page files	Find system files of a command	whereis java
xargs	Build and execute commands from input	Pipe results to another command	find . -name *.log xargs rm -f

3 FIND COMMAND EXAMPLES

```
$ find /home -name file.txt
# Find file.txt in /home

$ find / -type d -name logs
# Find directories named logs

$ find . -size +10M
# Find files larger than 10MB

$ find /var/log -mtime -7
# Files modified in last 7 days
```



4 GREP COMMAND EXAMPLES

```
$ grep "error" /var/log/syslog
# Search 'error' in syslog

$ grep -i "login" auth.log
# Case-insensitive search

$ grep -r "TODO" /home/user/project
# Recursive search in directory

$ grep -n "failed" log.txt
# Show line number with match
```

GREP OPTIONS

- i Ignore case
- r Recursive
- n Show line number
- v Invert match
- l Show filename only
- c Count matches

5 LOCATE COMMAND

```
$ locate filename
# Fast search using database

$ updatedb
# Update locate database
```

locate is very fast but depends on the database. Use updatedb regularly (cron).

6 WHICH & WHEREIS EXAMPLES

```
$ which gcc
# /usr/bin/gcc

$ which python3
# /usr/bin/python3
```

```
$ whereis gcc
# gcc: /usr/bin/gcc /usr/share/man/man1/gcc.1.gz

$ whereis ls
# ls: /bin/ls /usr/share/man/man1/ls.1.gz
```



7 PRACTICAL EXAMPLES

- ☆ \$find /etc -name "*.conf" # Find all .conf files in /etc
- ☆ \$find / -perm 644 -type f # Find files with permission 644
- ☆ \$grep -r "password" /etc/ # Search 'password' in /etc recursively
- ☆ \$locate hosts # Find hosts file quickly
- ☆ \$which docker # Find docker executable path
- ☆ \$whereis ssh # Find ssh binary and man page
- ☆ \$find /home -name "*.txt" -mtime -3 # .txt files modified in last 3 days



8 COMMON MISTAKES

- ✗ Using find without path (slow on large systems)
- ✗ Not updating locate database
- ✗ Forgetting -r with grep for recursive search
- ✗ Using rm with xargs without -r (can be dangerous)
- ✗ Not quoting patterns with spaces



9 QUICK TIPS

- ✓ Use find for deep and advanced search.
- ✓ Use locate for quick (name-based) search.
- ✓ Use grep to search inside file contents.
- ✓ Combine commands with pipes | for power.



MINI CHALLENGE

1. Find all .log files in /var/log.
2. Search for "failed" in all log files.
3. Find files larger than 50MB in /home.
4. Locate and display path of python3.

Try these in your terminal!

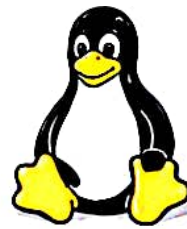
10 SUMMARY

Search is a superpower in Linux. Master these commands and you can find anything, anywhere!



6

LINUX FILE PERMISSIONS



In Linux, security starts with permissions!

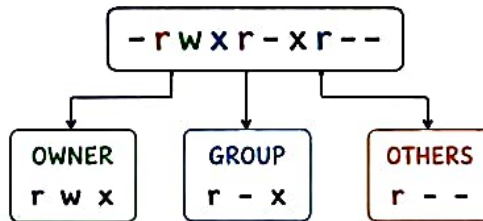
Control who can READ, WRITE and EXECUTE files and directories.

1 WHAT ARE PERMISSIONS?

- Every file and directory has an owner, a group and others.
- Each has permission to:
 - ✓ Read (r) → view the content
 - ✓ Write (w) → modify or delete
 - ✓ Execute (x) → run or access



2 PERMISSION STRUCTURE



WHO ARE THEY?

- Owner → file owner
- Group → group owner
- Others → everyone else

3 PERMISSION VALUES

Permission	Symbol	Numeric Value	Meaning
Read	r	4	View file content
Write	w	2	Modify file content
Execute	x	1	Run file / access dir
-	-	0	No permission

Remember:

$$4 + 2 + 1 = 7 \text{ (rwx)} \quad 4 + 0 + 1 = 5 \text{ (r-x)}$$

$$4 + 2 + 0 = 6 \text{ (rw-)} \quad 0 + 0 + 0 = 0 \text{ (---)}$$

4 NUMERIC PERMISSIONS (EXAMPLES)

Numeric	Symbolic	Permission	Meaning
777	rwxrwxrwx	Full access	Owner, group & others can read, write, execute
755	rwxr-xr-x	Common for folders	Owner full access, others read & execute
644	rw-r--r--	Common for files	Owner read & write, others read only
600	rw-----	Private file	Only owner can read & write
700	rwx-----	Private folder/script	Only owner full access
400	r-----	Read only	Only owner can read
000	-----	No access	No one can do anything

5 CHECK PERMISSIONS

\$ ls -l file.txt

```

-rw-r--r-- 1 user group 1024 May 20 10:30 file.txt
  ↑      ↑      ↑      ↑      ↑      ↑      ↑
Permissions Links Owner Group Size Date Name
    
```

- First 10 characters show permissions.
- (first character) = file
 - d (first character) = directory



6 CHANGE PERMISSIONS WITH chmod

SYMBOLIC METHOD

\$ chmod u+x script.sh

- u → user (owner)
- g → group
- a → others
- a → all

+ → add - → remove = → set

NUMERIC METHOD

\$ chmod 755 script.sh

Set permissions using numbers

Tip: Use numeric method for faster workflow!



7 COMMON chmod EXAMPLES

```

$ chmod 755 file.sh # Owner rwx, group rx, others rx
$ chmod 644 file.txt # Owner rw, group r, others r
$ chmod 700 private.sh # Only owner can rwx
$ chmod 664 data.txt # Owner rw, group rw, others r
$ chmod -R 755 /folder # Recursive change for directory
    
```



8 SPECIAL PERMISSIONS (Advanced)

SUID (Set User ID)

On executables.
Run with file owner's privileges.

Example:

\$ chmod u+s prog (or 4755)

SGID (Set Group ID)

On executables or directories.
Run with group privileges.

Example:

\$ chmod g+s prog (or 2755)

Sticky Bit

On directories.
Only file owner can delete or rename files inside.

Example:

\$ chmod +t /tmp (or 1777)



9 PRACTICE TASKS

1. Create a file and a directory.
2. Set permissions 644 on the file.
3. Set permissions 755 on the directory.
4. Remove execute permission from others.
5. Try using symbolic and numeric methods.

Practice makes perfect! 😊

10 COMMON MISTAKES

- ✗ Giving 777 to everything (security risk)
- ✗ Forgetting execute permission on scripts
- ✗ Confusing file and directory permissions
- ✗ Not checking owner/group before changing
- ✗ Using chmod -R carelessly



11 SUMMARY

- ✓ Linux uses r, w, x permissions.
- ✓ Each has numeric value 4, 2, 1.
- ✓ Use ls -l to check permissions.
- ✓ Use chmod to change permissions.
- ✓ Understand special permissions.
- ✓ Secure system = strong permissions!



PRO TIP

Always follow the principle of least privilege - give only the permissions that are necessary, not more!

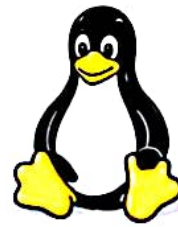


Security is not a product, it's a process.



7

LINUX USERS & GROUPS



In Linux, multiple users can work on the same system securely!

Manage who can access the system and what they can do.

1 WHY USERS & GROUPS?

- Multi-user system → Many people can use Linux at the same time.
- Security → Each user has own identity and permissions.
- Organization → Groups help manage users easily.
- Control access → Limit what users can do.



2 USERS vs GROUPS

USERS

- Individual account to access system.
- Has unique UID.
- Own home directory.



GROUPS

- Collection of users.
- Has unique GID.
- Simplifies permission management.



KEY FILES

/etc/passwd → User info
/etc/shadow → Passwords
/etc/group → Group info
/etc/gshadow → Group passwords



3 IMPORTANT COMMANDS

COMMAND	PURPOSE	EXAMPLE
useradd	Create a new user	\$ sudo useradd john
passwd	Set or change user password	\$ sudo passwd john
userdel	Delete a user	\$ sudo userdel john
usermod	Modify user properties	\$ sudo usermod -aG dev john
groupadd	Create a new group	\$ sudo groupadd dev
groupdel	Delete a group	\$ sudo groupdel dev
id	Show user & group ID	\$ id john
groups	Show groups of user	\$ groups john

4 USER INFORMATION

```
$ id john
uid=1001(john) gid=1001(john)
groups=1001(john), 1002(dev), 1003(sudo)
```

↓ ↓ ↓
User ID Primary Group ID Other Groups



Each user has:

- UID (User ID)
- Primary GID (User's main group)
- Additional groups (for permissions)

5 GROUP INFORMATION

```
$ getent group dev
dev:x:1002:john,alice,bob
```

↑ ↑ ↑
Group Name GID Members

6 EXAMPLES

```
$ sudo useradd alice           # Create user alice
$ sudo passwd alice           # Set password for alice
$ sudo useradd -m -s /bin/bash bob # Create user bob with home & shell
$ sudo userdel charlie         # Delete user charlie

$ sudo groupadd dev            # Create group dev
$ sudo usermod -aG dev john    # Add user john to group dev
$ sudo usermod -aG dev alice   # Add user alice to group dev
$ groups john                  # View groups of john
```

7 SWITCH USER

COMMAND	PURPOSE
su username	Switch to another user (need password)
su - username	Switch to another user with full environment
sudo command	Run command as another user (admin)

Example

```
$ su john
$ su - john
$ sudo ls /root
```

8 SUDO (SUPERUSER DO)

- Allows permitted users to run commands as root.
- Check sudo access:

```
$ sudo -l
(Lists allowed commands)
```

Example

```
$ sudo systemctl restart nginx
$ sudo apt update
```



9 BEST PRACTICES

- ★ Create normal users, not root.
- ★ Use groups to manage permissions.
- ★ Use sudo instead of root login.
- ★ Regularly review users and groups.



10 COMMON MISTAKES

- ✗ Working as root user.
- ✗ Giving users unnecessary sudo access.
- ✗ Not removing old users or groups.
- ✗ Weak passwords.



11 PRACTICE TASKS

- Create a user testuser.
- Set password for testuser.
- Create a group team.
- Add testuser to team group.
- Check groups of testuser.
- Switch to testuser and create a file.

12 SUMMARY

- ✓ Users = Individual accounts.
- ✓ Groups = Collection of users.
- ✓ Permissions are controlled using users & groups.
- ✓ Use sudo for administrative tasks.
- ✓ Security + Organization = Strong System!



PRO TIP

Learn the principle of "least privilege" → Give only the access that is needed, not more!



8

LINUX USERS & GROUPS

Manage access. Protect your system.



Linux is multi-user by design. Good user management = Secure system!

1 WHAT ARE USERS & GROUPS?



USERS

An user represents a person who can log in and use the system.



GROUPS

A group is a collection of users with common permissions.



Permissions can be assigned to users or groups.

2 TYPES OF USERS



ROOT USER

Superuser with unlimited power.

UID = 0
Be careful with root!



REGULAR USER

Normal users for daily work.

UID = 1000+
Least privilege principle.



SYSTEM USER

Created by system for services.

UID = 1-999
Cannot login normally.



3 USER MANAGEMENT COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
useradd	Create a new user	sudo useradd john
usermod	Modify an existing user	sudo usermod -aG dev john
userdel	Delete a user	sudo userdel john
passwd	Set or change password	sudo passwd john
id	Show user ID and groups	id john
whoami	Show current user	whoami
su	Switch user	su - john

4 GROUP MANAGEMENT COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
groupadd	Create a new group	sudo groupadd dev
groupmod	Modify a group	sudo groupmod -n developers dev
groupdel	Delete a group	sudo groupdel dev
groups	Show groups of user	groups john

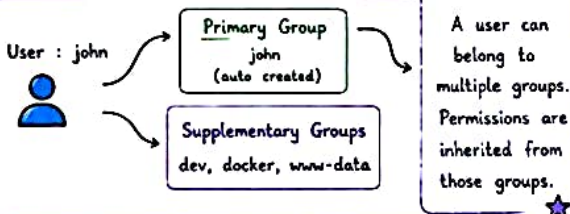


Primary group is created automatically when a new user is added.



-G : Primary group -aG : Add to supplementary group
-M : Create user without home -s : Set shell

5 USER & GROUP RELATIONSHIP



6 CHECK USERS & GROUPS

```
$ cat /etc/passwd # List all users
$ cat /etc/group  # List all groups
$ id <user>       # Show user ID, groups
$ groups <user>   # Show groups of user
$ getent passwd   # Another way to list users
$ getent group    # Another way to list groups
```

Example Output (id john)
uid=1001(john) gid=1001(john)
groups=1001(john),
1002(dev), 1003(docker)

7 FILE OWNERSHIP

Every file has 3 owners:

- User (u) - Owner
- Group (g) - Group owner
- Others (o) - Everyone else

Check ownership

```
$ ls -l file.txt
-rw-r--r-- 1 john dev 1234 May 20 10:30 file.txt
  ↑      ↑      ↑      ↑      ↑      ↑      ↑
Permissions Links Owner Group Size Date Name
```

8 CHANGE OWNERSHIP & GROUP

Change Owner

```
$ chown user file.txt # Change owner
$ chown user:group file.txt # Change owner & group
$ chown -R user:group /dir # Recursive change
```

Change Group

```
$ chgrp group file.txt # Change group
$ chgrp -R group /dir # Recursive change
```

Tip

Use -R carefully!
It changes files in all subdirectories.

9 BEST PRACTICES

- ✓ Create users for real people, not root.
- ✓ Use groups to manage permissions easily.
- ✓ Apply least privilege - give only what's needed.
- ✓ Regularly review users and remove unused accounts.
- ✓ Use strong passwords and lock unused accounts.

SUMMARY

- ★ Users authenticate and access the system.
- ★ Groups simplify permission management.
- ★ Use proper commands to create, modify, and manage users & groups.
- ★ Always follow security best practices!

QUICK CHEAT SHEET

```
useradd john
passwd john
usermod -aG dev john
userdel john
groupadd dev
chown user:group file
```



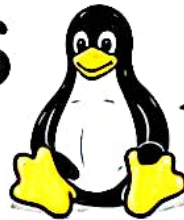
PRO TIP

Good user & group management is the foundation of system security!



9

LINUX PROCESSES MANAGE & CONTROL



A system is only as good as the processes it runs.

Every program you run is a process.
Learn to view, manage and control them!

1 WHAT IS A PROCESS?

- A process is a running instance of a program.
- Each process has a unique PID (Process ID).
- Processes use system resources like CPU, memory, files, etc.
- Linux is a multi-tasking system.



2 VIEW PROCESSES

COMMAND	DESCRIPTION	EXAMPLE
ps	Show running processes	ps aux
top	Real-time process viewer	top
htop	Interactive process viewer (better than top)	htop
pstree	Show processes in tree form	pstree
pgrep	Search process by name	pgrep nginx
pidof	Show PID of a process	pidof sshd

ps aux output meaning

USER - Owner of process
PID - Process ID
%CPU - CPU usage
%MEM - Memory usage
VSZ - Virtual memory
RSS - Resident memory
STAT - Process status
START - Start time
TIME - CPU time used
COMMAND - Command name

3 PROCESS STATES (STAT COLUMN)

R (Running)	Currently running or runnable
S (Sleeping)	Sleeping (waiting for event)
D (Uninterruptible Sleep)	Waiting for I/O (usually disk)
T (Stopped)	Stopped by job control signal
Z (Zombie)	Process terminated but not reaped by parent
< (High Priority)	High priority process
N (Low Priority)	Low priority process
s (Session Leader)	Process is session leader
l (Multi-threaded)	Process has multiple threads

4 COMMON PROCESS OPERATIONS

TASK	COMMAND	EXAMPLE
Kill a process	kill PID	kill 1234
Kill with signal	kill -SIGNAL PID	kill -9 1234
Graceful stop	kill -15 PID (SIGTERM)	kill -15 1234
Force kill	kill -9 PID (SIGKILL)	kill -9 1234
Kill by name	pkill process_name	pkill nginx
Kill by pattern	pkill -f pattern	pkill -f python
Kill all by name	killall process_name	killall chrome
Change priority	nice -n value command	nice -n 10 python app.py
Renice running	renice value -p PID	renice 5 -p 1234

SIGNALS YOU SHOULD KNOW

- SIGHUP - Reload
- SIGINT - Interrupt
- SIGKILL - Force kill
- SIGTERM - Terminate
- SIGSTOP - Stop
- SIGCONT - Continue

Always try SIGTERM (15) first. Use SIGKILL (9) only when necessary.

5 JOB CONTROL (IN TERMINAL)

- Run a command → creates a job.
- Jobs can be foreground or background.
- Useful for managing long running tasks.



COMMAND	DESCRIPTION	EXAMPLE
&	Run in background	sleep 100 &
jobs	List background jobs	jobs
fg %job	Bring job to foreground	fg %1
bg %job	Resume in background	bg %1
Ctrl + Z	Suspend current job	(press keys)

Example Workflow
1. long-task &
2. jobs
3. fg %1
4. Ctrl+Z
5. bg %1

6 MONITORING PROCESS RESOURCES

COMMAND	DESCRIPTION	EXAMPLE
top	Live CPU & memory usage	top
htop	Better interactive monitor	htop
ps aux --sort=-%cpu	Sort by CPU usage	ps aux --sort=-%cpu head
ps aux --sort=-%mem	Sort by Memory usage	ps aux --sort=-%mem head
free -h	Check memory usage	free -h
vmstat 1 5	System performance stats	vmstat 1 5
iostat -x 1 5	Disk I/O stats	iostat -x 1 5

7 REAL LIFE EXAMPLES

- Find nginx process and kill it gracefully
\$ pgrep nginx
\$ kill -15 \$(pgrep nginx)
- Force kill a hung process
\$ kill -9 1234
- Run a backup script in background
\$./backup.sh &
- Check top 10 CPU consuming processes
\$ ps aux --sort=-%cpu | head



8 COMMON MISTAKES

- Using kill -9 immediately
- Killing critical system processes (can crash your system)
- Not checking what process does
- Running too many background jobs
- Forgetting zombie processes (check with ps aux | grep Z)



9 SUMMARY

- Every program is a process.
- Use ps, top, htop to view processes.
- Use kill, pkill, killall to manage them.
- Understand signals before killing.
- Monitor resources and optimize.
- Good process management = a healthy system!



PRO TIP

Learn to read process info like a pro.
It helps in troubleshooting, performance tuning, and system administration.



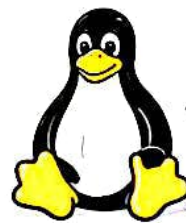
Practice daily, become better!



10

LINUX PACKAGE MANAGEMENT

Install, update, **remove** and manage software easily!



Packages make your life easy. The package manager makes it effortless!

1 WHAT IS A PACKAGE?

- A package is a software bundled with all the files it needs to run.
- Package managers automate installation, updates, dependency handling, and removal.
- Different Linux distributions use different package managers.



2 COMMON PACKAGE MANAGERS

DISTRIBUTION FAMILY	EXAMPLES	PACKAGE MANAGER	COMMAND STYLE
Debian / Ubuntu (APT)	Ubuntu, Debian, Linux Mint	apt, apt-get	apt install pkg
Red Hat / CentOS (YUM/DNF)	RHEL, CentOS, Rocky, AlmaLinux	yum (old), dnf (new)	dnf install pkg
Arch Linux (Pacman)	Arch, Manjaro	pacman	pacman -S pkg
openSUSE (Zypper)	openSUSE Leap	zypper	zypper install pkg

3 APT (DEBIAN / UBUNTU)



APT is the most commonly used package manager in Ubuntu and Debian based systems.

TASK	COMMAND	EXAMPLE
Update package index	sudo apt update	sudo apt update
Upgrade installed packages	sudo apt upgrade	sudo apt upgrade
Install a package	sudo apt install <pkg>	sudo apt install nginx
Remove a package	sudo apt remove <pkg>	sudo apt remove nginx
Purge (remove + config)	sudo apt purge <pkg>	sudo apt purge nginx
Search a package	apt search <keyword>	apt search docker
Show package info	apt show <pkg>	apt show git
List installed packages	dpkg -l	dpkg -l less
Clean cache	sudo apt clean	sudo apt clean
Autoremove unused pkgs	sudo apt autoremove	sudo apt autoremove

Tip: Always update the index before installing a new package.

4 DNF (RED HAT / CENTOS / FEDORA)



DNF is the next generation package manager for RHEL based distributions.

TASK	COMMAND	EXAMPLE
Update package index	sudo dnf check-update	sudo dnf check-update
Upgrade installed packages	sudo dnf upgrade	sudo dnf upgrade
Install a package	sudo dnf install <pkg>	sudo dnf install httpd
Remove a package	sudo dnf remove <pkg>	sudo dnf remove httpd
Search a package	dnf search <keyword>	dnf search mysql
Show package info	dnf info <pkg>	dnf info git
List installed packages	dnf list installed	dnf list installed
Clean cache	sudo dnf clean all	sudo dnf clean all
Autoremove unused pkgs	sudo dnf autoremove	sudo dnf autoremove

Tip: Use "dnf upgrade" regularly to keep your system secure.

5 PACMAN (ARCH LINUX / MANJARO)



Pacman is fast and simple (Arch Linux default).

TASK	COMMAND	EXAMPLE
Update package index	sudo pacman -Sy	sudo pacman -Sy
Upgrade system	sudo pacman -Syu	sudo pacman -Syu
Install a package	sudo pacman -S <pkg>	sudo pacman -S vim
Remove a package	sudo pacman -R <pkg>	sudo pacman -R vim
Search a package	pacman -Ss <keyword>	pacman -Ss python
Show package info	pacman -Si <pkg>	pacman -Si nginx
List installed packages	pacman -Q	pacman -Q
Remove unused deps	sudo pacman -Rns	sudo pacman -Rns
Clean package cache	sudo pacman -Sc	sudo pacman -Sc

Tip: -S = Sync & Install, -R = Remove, -Q = Query

6 ZYPPER (OPENSUSE)



Zypper is used in openSUSE distributions.

TASK	COMMAND	EXAMPLE
Refresh repositories	sudo zypper refresh	sudo zypper refresh
Install a package	sudo zypper install <pkg>	sudo zypper install vim
Remove a package	sudo zypper remove <pkg>	sudo zypper remove vim
Search a package	zypper search <keyword>	zypper search docker
Show package info	zypper info <pkg>	zypper info nginx
List installed packages	zypper search --installed-only	zypper search -i
Update packages	sudo zypper update	sudo zypper update
Clean cache	sudo zypper clean	sudo zypper clean
Autoremove unused pkgs	sudo zypper autoremove	sudo zypper autoremove

Tip: Use "refresh" before installing or searching.

7 COMMON PACKAGE SOURCES



Official Repositories
Safe, tested and supported by the distribution.



PPAs (Ubuntu)
Personal Package Archives (add extra software).



EPEL / RPM Fusion
Extra repositories for RHEL/Fedora based systems.



AUR (Arch Linux)
Community repository for user contributed packages.

Always prefer official repositories for stability and security.



8 BEST PRACTICES

- Update the package index regularly.
- Upgrade your system frequently.
- Remove unused packages and dependencies.
- Use official repositories whenever possible.
- Read package descriptions before installing.
- Backup important data before major updates.
- Keep your system clean and secure!



9 SUMMARY



Packages = software bundles.



Package managers handle install, update, remove & dependencies.



APT for Debian/Ubuntu.



DNF for RHEL/CentOS/Fedora.



Pacman for Arch Linux.



Zypper for openSUSE.

• Know your tool. Use it well!



PRO TIP



@NeerajSingh-DevOps

